Technische Hochschule Ingolstadt

Fakultät für Elektrotechnik und Informatik

Zukunft in Bewegung

Reaktive Programmierung

Dokumenthistorie

Autor: Prof. Dr. Bernd Hafenrichter

Version: 1.0

Letze Änderung: 15.04.2025

Motivation

Problemstellung - Thread per Request Model

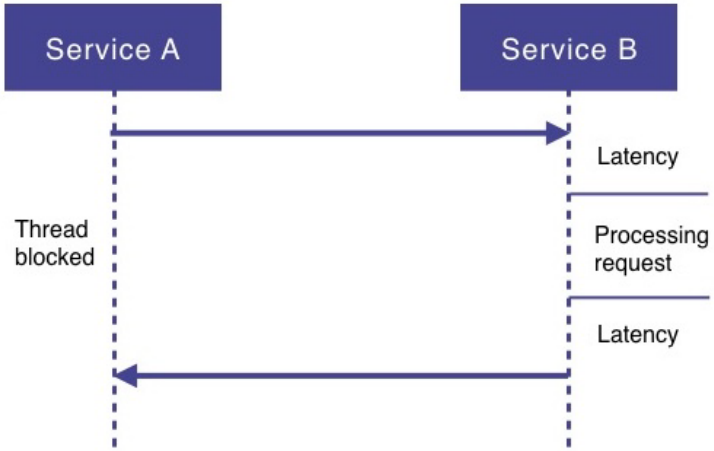
- Klassische Applikationen arbeiten auf dem Thread-per-Request-Modell.
- D.h. jede Anfrage wird durch einen einzelnen Thread bearbeitet
- Das bedeutet, dass die Anwendung nur so viele gleichzeitige Anfragen bearbeiten kann, wie der Threadpool groß ist.

https://dzone.com/articles/an-introduction-to-reactive-programming-with-sprin

Motivation

Problemstellung - Waiting for I/O Operations

- Ein Thread muss auf den Abschluss einzelnen I/O-Operationen warten.
- In all diesen Situationen wird der Thread, der die E/A-Anfrage stellt, blockiert und wartet untätig, bis die E/A-Operation abgeschlossen ist.
- Die verfügbaren Threads werden nicht ausgenutzt. Dies ist eine verschwendung von Ressourcen.

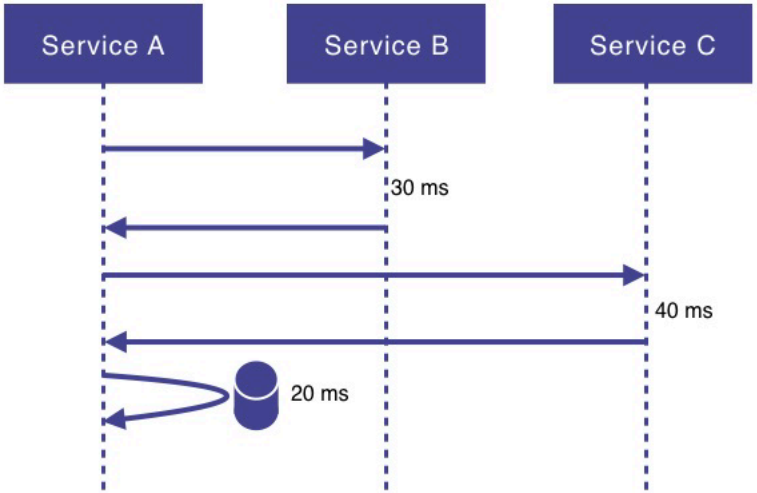


https://dzone.com/articles/an-introduction-to-reactive-programming-with-sprin

Motivation

Problemstellung - Response Times

- Mehre IO-Operationen die nacheinander ausgeführt werden führen ebenfalls zu langen Wartezeiten.
- Dies ist dann problematisch wenn es keinen fachlichen Grund für die sequentielle Ausführung gibt.



https://dzone.com/articles/an-introduction-to-reactive-programming-with-sprin

Motivation

Problemstellung - Overwhelming the Client

- Ruft ein Client eine sehr große Datenmenge ab, kann es z.B. zu Speicherproblemen kommen, da die gesamte Antwortnachricht auf Server-Seite aufgebaut werden muss bevor diese an den Client zurückgegeben wird.
- Als Folge können Fehler z.B. durch Out-Of-Memory-Probleme auftreten

https://dzone.com/articles/an-introduction-to-reactive-programming-with-sprin

Motivation

Reaktive Programmierung

- Reaktive Programmierung versucht die beschriebenen Probleme zu lösen
- Folgend Vorteile ergeben sich dadurch
  - Abkehr vom Modell „Thread pro Anfrage“ und Bearbeitung von mehr Anfragen mit einer geringen Anzahl von Threads möglichst gleichzeitig.
  - Verhindern, dass Threads blockieren, während sie auf den Abschluss von E/A-Vorgängen warten
  - Parallele Anrufe leicht machen
  - Unterstützung von „Backpressure“, so dass der Client die Möglichkeit hat, dem Server mitzuteilen, wie viel Last er bewältigen kann

https://dzone.com/articles/an-introduction-to-reactive-programming-with-sprin

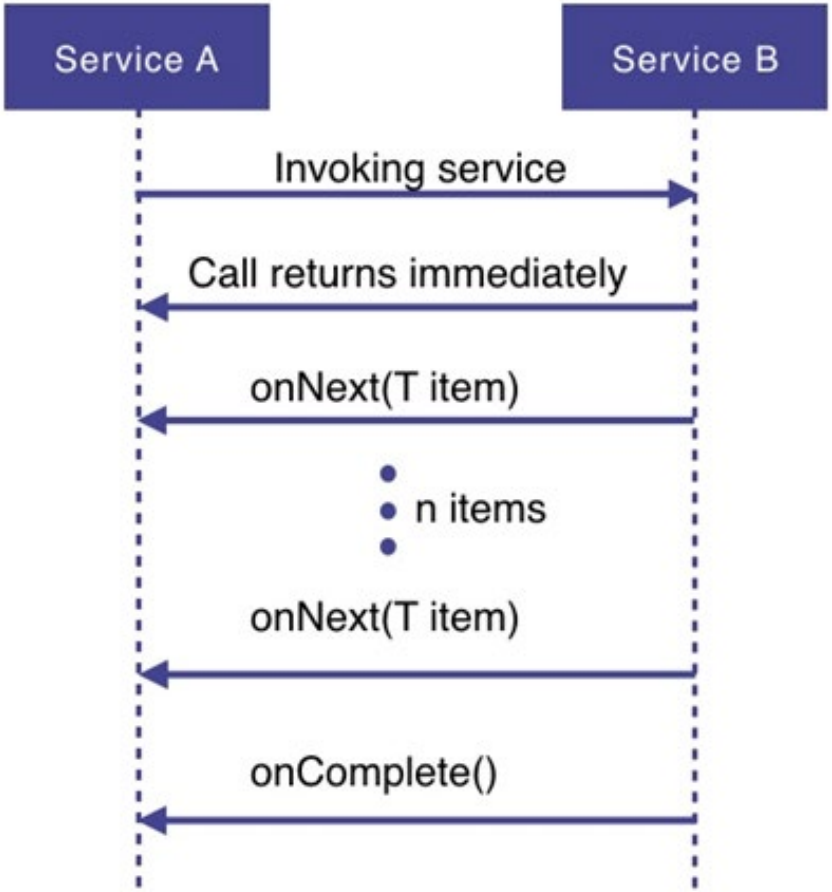
Definition

Reaktive Applikation

„Bei der reaktiven Programmierung um nicht blockierende Anwendungen, die asynchron und ereignisgesteuert sind und eine geringe Anzahl von Threads benötigen, um zu skalieren. Ein wichtiger Aspekt dieser Definition ist das Konzept des Backpressure, ein Mechanismus, der sicherstellt, dass die Produzenten die Konsumenten nicht überfordern.“

https://dzone.com/articles/an-introduction-to-reactive-programming-with-sprin

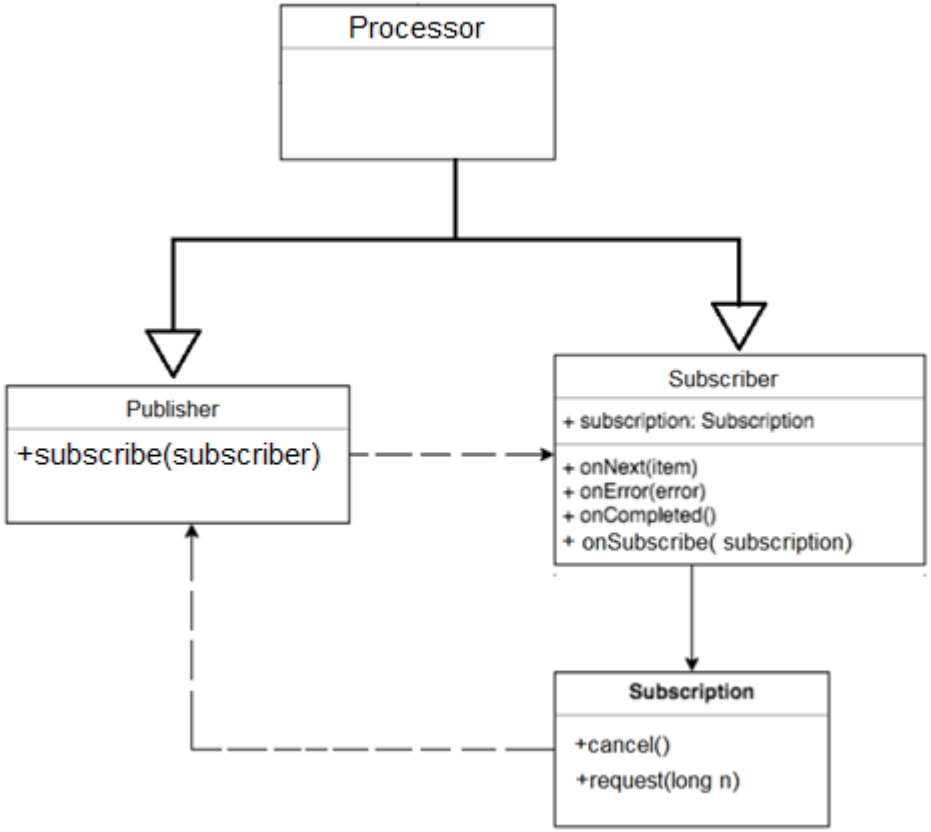
Die Grundidee



- Service A stehlt eine Anfrage an Service B
- Service-B startet die Verarbeitung, der Aufruf kehrt aber sofort zurück.
- Das Ergebnis wird als Data-Stream bereitgestellt.
- Immer wenn Service B ein Ergebnis produziert hat, wird dieses an Service-A zurück gegeben
- Sind alle Daten verarbeitet wird der onComplete-Event geliefert
- Im Falle eines Fehlers wird dieser ebenfalls an den Client übermittelt

https://dzone.com/articles/an-introduction-to-reactive-programming-with-sprin

Die Java-Flow-Spezifikation



https://docs.oracle.com/javase/9/docs/api/java/util/concurrent/Flow.html

https://dzone.com/articles/an-introduction-to-reactive-programming-with-sprin

Die Java-Flow-Spezifikation

Publisher

- Dieser stellt den Datenproduzenten/die Datenquelle dar und verfügt über eine Methode, mit der sich der Abonnent beim Herausgeber anmelden kann.

Java

```java
public interface Publisher<T> {

    public void subscribe(Subscriber<? super T> s);

}
```

https://docs.oracle.com/javase/9/docs/api/java/util/concurrent/Flow.html

https://dzone.com/articles/an-introduction-to-reactive-programming-with-sprin

Hintergrund

Subscriber

- Dieser stellt den Verbraucher dar und verfügt über die folgenden Methoden:

Java

```java
public interface Subscriber<T> {

    public void onSubscribe(Subscription s);

    public void onNext(T t);

    public void onError(Throwable t);

    public void onComplete();

}
```

https://docs.oracle.com/javase/9/docs/api/java/util/concurrent/Flow.html

https://dzone.com/articles/an-introduction-to-reactive-programming-with-sprin

Hintergrund

Subscription

- Die Abonnements enthalten Methoden, die es dem Subscriber ermöglichen, die Ausgabe von Elementen durch den Publisher zu kontrollieren (d. h. Unterstützung von Backpressure).

Java

```java
public interface Subscription {

    public void request(long n);

    public void cancel();

}
```

https://docs.oracle.com/javase/9/docs/api/java/util/concurrent/Flow.html

https://dzone.com/articles/an-introduction-to-reactive-programming-with-sprin

Hintergrund

Processor

- Wenn eine Entität eingehende Elemente umwandeln und dann an einen anderen Abonnenten weitergeben soll, ist eine Implementierung der Schnittstelle Processor erforderlich. Diese fungiert sowohl als Subscriber als auch als Publisher.

Java

```java
public interface Processor<T, R> extends Subscriber<T>, Publisher<R> {

}
```

https://docs.oracle.com/javase/9/docs/api/java/util/concurrent/Flow.html

https://dzone.com/articles/an-introduction-to-reactive-programming-with-sprin

The reactive principle

- In The Reactive Principle, dem Nachfolger von The Reactive Manifesto, definieren Jonas Bonér et al. die acht Prinzipien, die eine Anwendung verkörpern muss, um als reaktiv zu gelten:

https://www.reactiveprinciples.org/principles/accept-uncertainty.html

The eight - reactive principle

- Reaktionsschnell bleiben. Reagieren Sie immer zeitnah und rechtzeitig.
- Unsicherheit akzeptieren. Bauen Sie Zuverlässigkeit trotz unzuverlässiger Grundlagen auf.
- Fehler akzeptieren. Rechnen Sie damit, dass etwas schief geht, und bauen Sie Widerstandsfähigkeit auf.
- Autonomie sichern. Entwickeln Sie Komponenten, die unabhängig voneinander agieren und zusammenarbeiten.

https://www.reactiveprinciples.org/principles/accept-uncertainty.html

The eigth - reactive principle

- Maßgeschneiderte Konsistenz. Individualisieren Sie die Konsistenz pro Komponente, um ein Gleichgewicht zwischen Verfügbarkeit und Leistung herzustellen.
- Zeit entkoppeln. Verarbeiten Sie asynchron, um Koordination und Wartezeiten zu vermeiden.
- Raum entkoppeln. Schaffen Sie Flexibilität, indem Sie das Netzwerk einbeziehen und die Verteilung einer Komponenten über mehrere Systeme ermöglichen.
- Dynamik handhaben. Passen Sie sich kontinuierlich an wechselnde Anforderungen und Ressourcen an.

https://www.reactiveprinciples.org/principles/accept-uncertainty.html

RxJAva – Ein Framework für reaktive Programmierung

- RxJava ist eine Java-VM-Implementierung für Reactive Applikationen
- Eine Bibliothek zur Erstellung von asynchroner und ereignisbasierter Programme unter Verwendung von event streams
- Es erweitert das Beobachtermuster, um Sequenzen von Daten/Ereignissen zu unterstützen, und fügt Operatoren hinzu, mit denen Sie Sequenzen deklarativ zusammenstellen können,
- Java RxJx versteckt Aspekte wie Low-Level-Threading, Synchronisierung, Thread-Sicherheit und nebenläufige Datenstrukturen.

https://github.com/ReactiveX/RxJava?tab=readme-ov-file

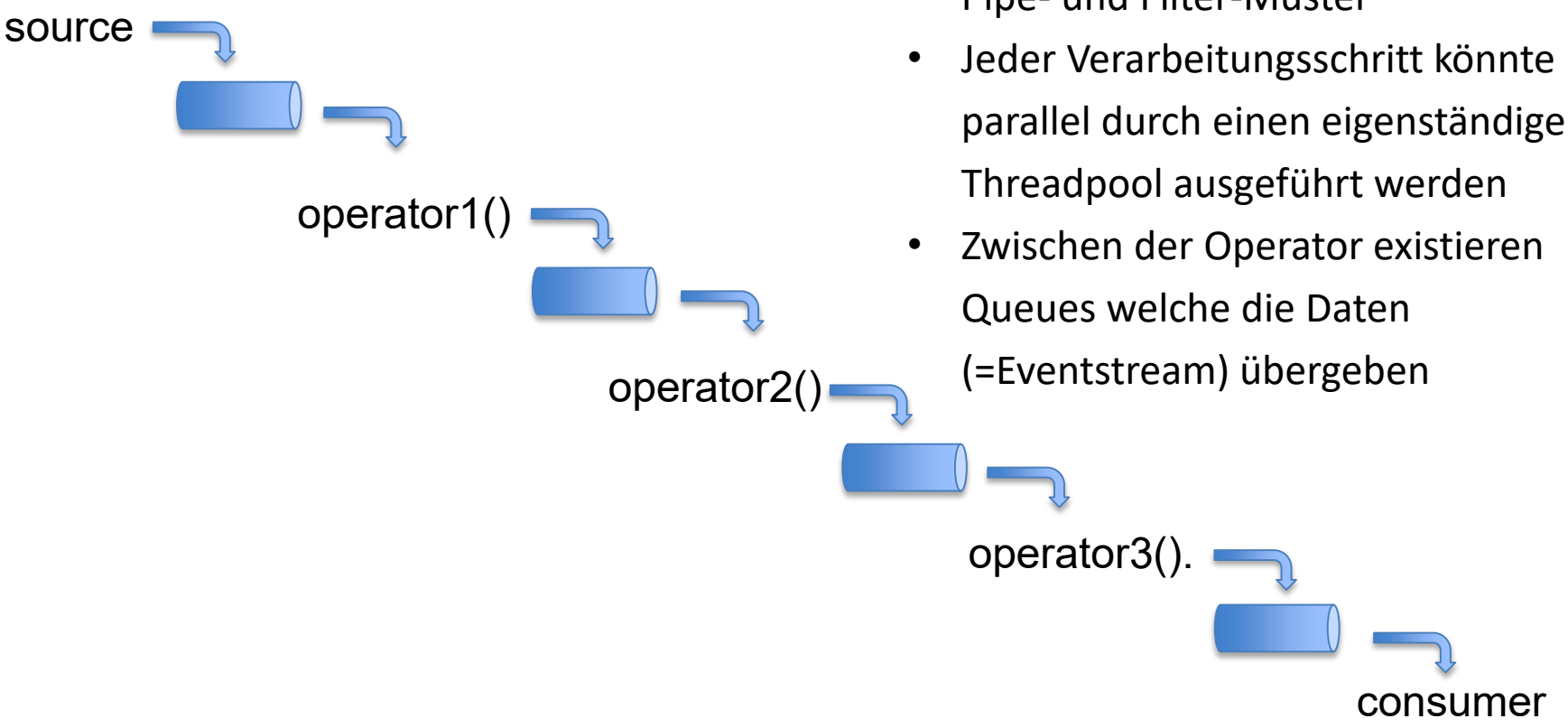
RxJAva – Ein Framework für reaktive Programmierung

- Die Datenflüsse in RxJava bestehen aus einer Quelle, keinem oder mehreren Zwischenschritten, gefolgt von einem Consumer.
  RxJava basiert auf FluentInterfaces welche das Zusammenstellen einer reaktiven Sequenz vereinfachen.

```
source.operator1()
      .operator2()
      .operator3()
      .subscribe(consumer);
```

https://github.com/ReactiveX/RxJava?tab=readme-ov-file

RxJAva – Ein Framework für reaktive Programmierung



- Die grundlegende Idee der Verarbeitung ist vergleichbar mit dem Pipe- und Filter-Muster
- Jeder Verarbeitungsschritt könnte parallel durch einen eigenständigen Threadpool ausgeführt werden
- Zwischen der Operator existieren Queues welche die Daten (=Eventstream) übergeben

RxJAva – Ein Framework für reaktive Programmierung

Assembly Time

- Die Vorbereitung der Datenflüsse durch Anwendung verschiedener Zwischenoperatoren erfolgt in der so genannten **assembly time**:
- Zu diesem Zeitpunkt sind die Daten noch nicht im Fluss, und es treten noch keine Nebeneffekte auf.

Subscription time

- Dies ist ein vorübergehender Zustand, wenn subscribe() auf einem Stream aufgerufen wird, der die Kette der Verarbeitungsschritte intern aufbaut und startet:

```
Flowable.range(1, 5)                    // Assembly
.map(v -> v * v)                        // Assembly
.filter(v -> v % 3 == 0)                // Assembly
.subscribe(System.out::println);        // Subscription
```

https://github.com/ReactiveX/RxJava?tab=readme-ov-file

RxJAva – Ein Framework für reaktive Programmierung

Runtime

- Dies ist der Zustand, in dem die Stream aktiv Daten, Ereignisse, Fehler oder Beendigungssignale aussenden:

```
Flowable<Integer> flow = Flowable.range(1, 5)
    .map(v -> v * v)
    .filter(v -> v % 3 == 0)
    .subscribe(System.out::println);
```

https://github.com/ReactiveX/RxJava?tab=readme-ov-file

RxJAva – Ein Framework für reaktive Programmierung

Operators

- Mit Hilfe von Operatoren können die Objekte eines Datenstroms in eine andere Darstellung übersetzt werden
- Mit Hilfe von Filter-Operatoren können die Objekte innerhalb eines Datenstroms gefiltert werden

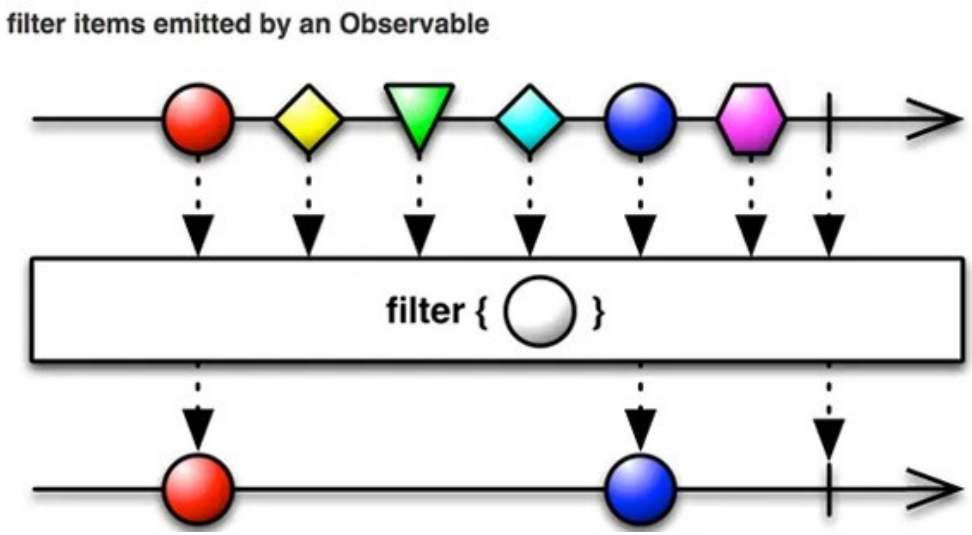
```
Flowable<Integer> flow = Flowable.range(1, 5)
    .map(v -> v * v)
    .filter(v -> v % 3 == 0)
    .subscribe(System.out::println);
```

https://github.com/ReactiveX/RxJava?tab=readme-ov-file

RxJAva – Ein Framework für reaktive Programmierung

Der Filter-Operator

- Mit Hilfe von Filtern können die Objekte innerhalb eines Datenstroms gefiltert werden
- Nachfolgend ist der filter-Operator dargestellt.

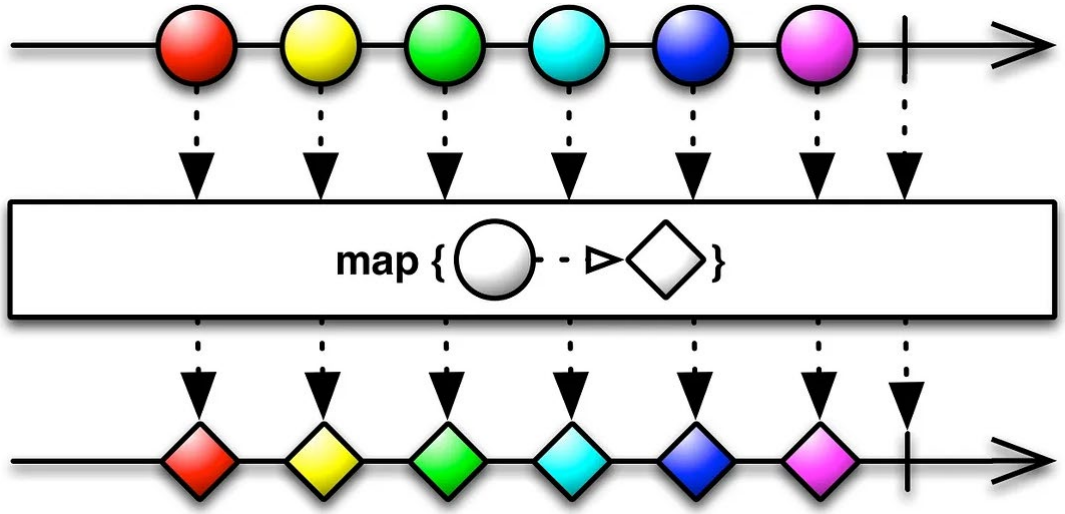


https://medium.com/@paroskwan/rxjava2-map-vs-flatmap-3e2e110cd7c8

RxJAva – Ein Framework für reaktive Programmierung

Der Map-Operators

- Der map-Operator ist für das übersetzen eines Ereignisses in eine andere Darstellung verantwortlich
- Veranschaulicht wird dieses Verhalten durch das nachfolgend Marble-Diagramm

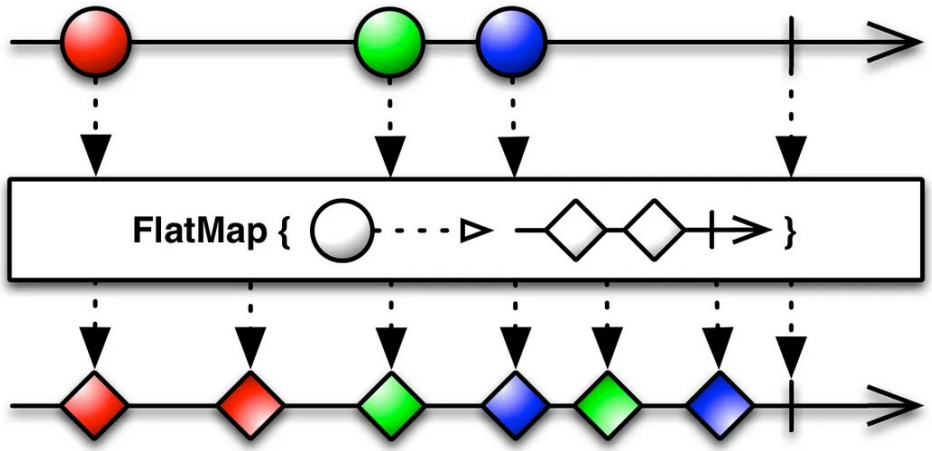


https://medium.com/@paroskwan/rxjava2-map-vs-flatmap-3e2e110cd7c8

RxJAva – Ein Framework für reaktive Programmierung

Der FlatMap-Operator

- Der flatMap-Operator übersetzt jedes Element, das von einer Datenstrom empfangen wird, mithilfe der bereitgestellten Funktion in einen neuen Datenstrom.
- Die Menge der erzeugten Ergebnisse wird wieder zu einem einzigen Datenstrom zusammengfasst.



https://medium.com/@paroskwan/rxjava2-map-vs-flatmap-3e2e110cd7c8

RxJAva – Ein Framework für reaktive Programmierung

Alle von RxJava angebotenen Operatoren

- ***Filtering Operators:*** Operators, which are used to selectively emit items from an Observable: Debounce, Distinct, ElementAt, Filter, First, IgnoreElements, Last, Sample, Skip, SkipLast, Take, and TakeLast.
- ***Transforming Operators:*** Operators that are used to transform an item emitted from an Observable: Buffer, FlatMap, GroupBy, Map, Scan, and Window.
- ***Utility Operators:*** Operators which are often useful with Observables: Delay, Materialize/Dematerialize, ObserveOn, Serialize, Subscribe, SubscribeOn, TimeInterval, Timeout, Timestamp, and Using.
- ***Mathematical Operators:*** Operators, which operates on whole items emitted by an Observable: Average, Concat, Count, Max, Min, Reduce, and Sum.

https://medium.com/@paroskwan/rxjava2-map-vs-flatmap-3e2e110cd7c8

RxJAva – Ein Framework für reaktive Programmierung

Advanced Operators

- ***Combining Operators:*** Operators that are used to create a single Observable from multiple Observables: And/Then/When, CombineLatest, Join, Merge, StartWith, Switch, and Zip.
- ***Conditional Operators:*** Operators that evaluates one or multiple Observables or items emitted: All, Amb, Contains, DefaultIfEmpty, SequenceEqual, SkipUntil, SkipWhile, TakeUntil, and TakeWhile.

https://medium.com/@paroskwan/rxjava2-map-vs-flatmap-3e2e110cd7c8

RxJAva – Ein Framework für reaktive Programmierung

Schedulers and background Computation

- RxJava-Operatoren arbeiten nicht direkt mit Threads oder ExecutorServices, sondern mit so genannten Schedulern
- Die Scheduler abstrahieren Quellen es Multithreading hinter einer einheitlichen API.
- RxJava 3 bietet mehrere Standard-Scheduler, die über die Utility-Klasse Schedulers zugänglich sind.
  - **Schedulers.computation():** Führen rechenintensive Arbeiten auf einer festen Anzahl von dedizierten Threads im Hintergrund aus. Die meisten asynchronen Operatoren verwenden dies als ihren Standard-Scheduler.
  - **Schedulers.io()**: Ausführung von E/A-ähnlichen oder blockierenden Operationen auf einer sich dynamisch ändernden Gruppe von Threads.
  - **Schedulers.single()**: Führen Sie die Arbeit auf einem einzigen Thread in einer sequentiellen und FIFO Weise aus.

https://github.com/ReactiveX/RxJava?tab=readme-ov-file

RxJAva – Ein Framework für reaktive Programmierung

Schedulers and background Computation

- Einer der häufigsten Anwendungsfälle für RxJava ist die Ausführung von Berechnungen oder Netzwerkanforderungen in einem Hintergrund-Thread und die Anzeige der Ergebnisse (oder Fehler) im UI-Thread:

```
Flowable.fromCallable(() -> {
    Thread.sleep(1000); //  imitate expensive computation
    return "Done";
})
  .subscribeOn(Schedulers.io())
  .observeOn(SwingScheduler.instance())
  .subscribe(System.out::println, Throwable::printStackTrace);
```

https://github.com/ReactiveX/RxJava?tab=readme-ov-file

RxJAva – Ein Framework für reaktive Programmierung

Backpressure

- Wenn der Datenfluss durch asynchrone Schritte läuft, kann jeder Schritt unterschiedliche Dinge mit unterschiedlicher Geschwindigkeit ausführen.
- Um zu vermeiden, dass solche Schritte überlastet werden, was sich in der Regel in einem erhöhten Speicherbedarf aufgrund von Zwischenpufferung oder der Notwendigkeit des Überspringens von Daten äußert, wird der so genannte Gegendruck angewandt, eine Form der Flusskontrolle, bei der die Schritte angeben können, wie viele Elemente sie zur Verarbeitung bereit sind.
- Dadurch kann die Speichernutzung der Datenströme in Situationen eingeschränkt werden, in denen ein Schritt im Allgemeinen nicht wissen kann, wie viele Elemente der vorgelagerte Schritt an ihn senden wird

https://github.com/ReactiveX/RxJava?tab=readme-ov-file